

МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт кибербезопасности и цифровых технологий
Кафедра КБ-4 «Интеллектуальные системы информационной
безопасности»

Отчет по практической работе № 6
по дисциплине «Специальные разделы математики»
Тема: «Прогнозирование временных рядов»

Выполнил: студент группы ББМО-01-25

Мухаметшин Александр Ринатович

Проверила:

Изотова Анастасия Андреевна

Москва, 2025

Ход работы

Задание 2. Прогнозирование с оптимальным параметром α

Условие:

Дан временной ряд температуры воздуха за 15 дней (в °C):

[18, 20, 19, 22, 24, 23, 25, 27, 26, 28, 30, 29, 31, 33, 32]

Требуется:

1. Найти оптимальное значение параметра α (перебором от 0.1 до 0.9 с шагом 0.1), минимизирующее MSE
2. Построить прогноз на 16-й день с найденным оптимальным α
3. Сравнить качество прогнозов для $\alpha = 0.2$ и оптимального α
4. Построить графики для обоих случаев

Решение:

```
import numpy as np
import matplotlib.pyplot as plt

plt.rcParams['font.family'] = 'DejaVu Sans'

# Исходные данные
temperature = [18, 20, 19, 22, 24, 23, 25, 27, 26, 28, 30, 29, 31, 33, 32]
print(f"\nВременной ряд температур: {temperature}")

def exponential_smoothing(data, alpha):
    n = len(data)
    smoothed = [0] * n
    forecasts = [0] * n
    errors = [0] * n

    smoothed[0] = data[0]
    forecasts[0] = data[0]

    for t in range(1, n):
        forecasts[t] = smoothed[t-1]
        errors[t] = data[t] - forecasts[t]
        smoothed[t] = alpha * data[t] + (1 - alpha) * smoothed[t-1]

    return smoothed, forecasts, errors

def calculate_mse(errors):
    return np.mean(np.array(errors[1:])**2)

# 1. Поиск оптимального  $\alpha$ 
print("\n1. Поиск оптимального  $\alpha$ :")
alphas = np.arange(0.1, 1.0, 0.1)
mse_values = []

for alpha in alphas:
```

```

_, _, errors = exponential_smoothing(temperature, alpha)
mse = calculate_mse(errors)
mse_values.append(mse)
print(f"α = {alpha:.1f}, MSE = {mse:.4f}")

optimal_alpha = alphas[np.argmin(mse_values)]
optimal_mse = min(mse_values)
print(f"\nОптимальное α = {optimal_alpha:.1f} с MSE = {optimal_mse:.4f}")

# 2. Прогноз на 16-й день с оптимальным α
smoothed_opt, forecasts_opt, errors_opt =
exponential_smoothing(temperature, optimal_alpha)
forecast_16 = smoothed_opt[-1]
print(f"\n2. Прогноз на 16-й день с α = {optimal_alpha:.1f}: {forecast_16:.2f}°C")

# 3. Сравнение с α = 0.2
smoothed_02, forecasts_02, errors_02 =
exponential_smoothing(temperature, 0.2)
mse_02 = calculate_mse(errors_02)
forecast_16_02 = smoothed_02[-1]

print(f"\n3. Сравнение α = 0.2 и оптимального α = {optimal_alpha:.1f}:")
print(f"α = 0.2: MSE = {mse_02:.4f}, Прогноз на 16-й день = {forecast_16_02:.2f}°C")
print(f"α = {optimal_alpha:.1f}: MSE = {optimal_mse:.4f}, Прогноз на 16-й день = {forecast_16:.2f}°C")

# 4. Графики
fig, axes = plt.subplots(2, 1, figsize=(6, 8))

# График для α = 0.2
axes[0].plot(temperature, 'o-', label='Факт', linewidth=2, markersize=8)
axes[0].plot(forecasts_02, 's--', label='Прогноз', linewidth=2, markersize=6)
axes[0].set_title(f'α = 0.2, MSE = {mse_02:.4f}')
axes[0].set_xlabel('День')
axes[0].set_ylabel('Температура (°C)')
axes[0].legend()
axes[0].grid(True, alpha=0.3)

# График для оптимального α
axes[1].plot(temperature, 'o-', label='Факт', linewidth=2, markersize=8)
axes[1].plot(forecasts_opt, 's--', label='Прогноз', linewidth=2, markersize=6)
axes[1].set_title(f'Оптимальное α = {optimal_alpha:.1f}, MSE = {optimal_mse:.4f}')
axes[1].set_xlabel('День')
axes[1].set_ylabel('Температура (°C)')

```

```
axes[1].legend()
axes[1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```

Вывод:

Временной ряд температур: [18, 20, 19, 22, 24, 23, 25, 27, 26, 28, 30, 29, 31, 33, 32]

1. Поиск оптимального α :

$\alpha = 0.1$, MSE = 35.4894
 $\alpha = 0.2$, MSE = 17.5339
 $\alpha = 0.3$, MSE = 10.2986
 $\alpha = 0.4$, MSE = 6.9405
 $\alpha = 0.5$, MSE = 5.2088
 $\alpha = 0.6$, MSE = 4.2569
 $\alpha = 0.7$, MSE = 3.7240
 $\alpha = 0.8$, MSE = 3.4387
 $\alpha = 0.9$, MSE = 3.3104

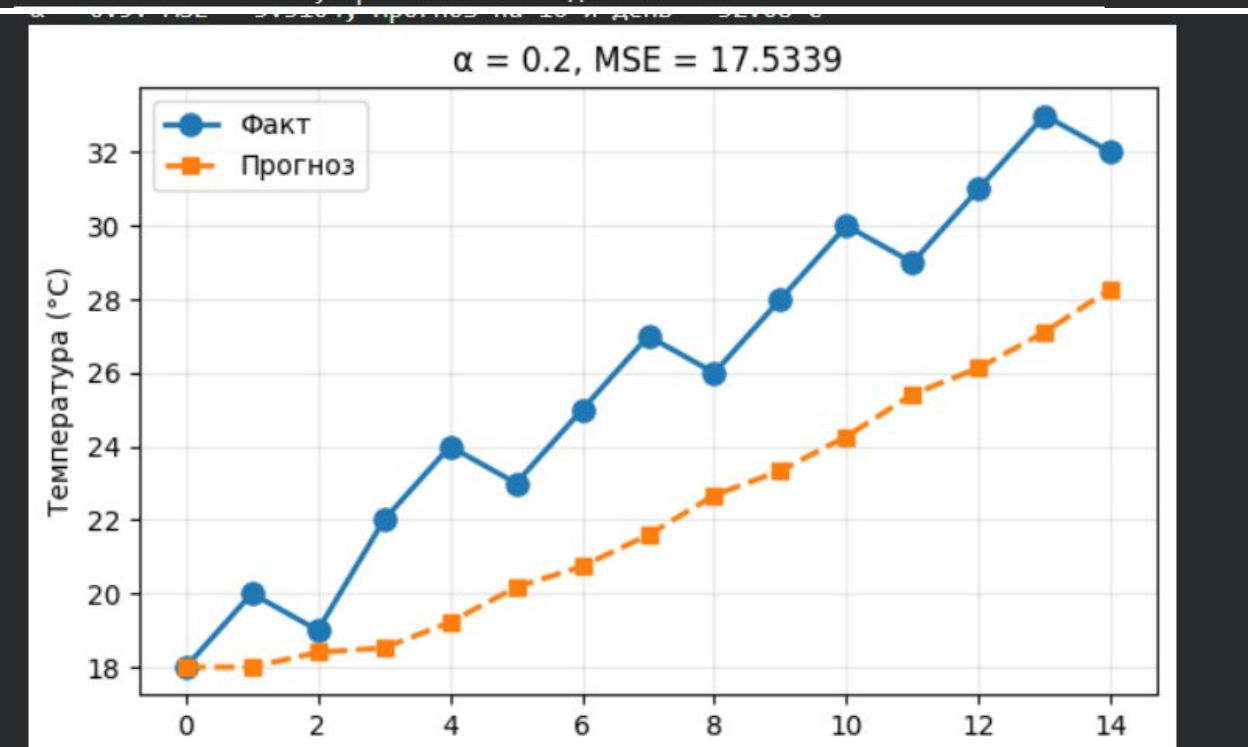
Оптимальное $\alpha = 0.9$ с MSE = 3.3104

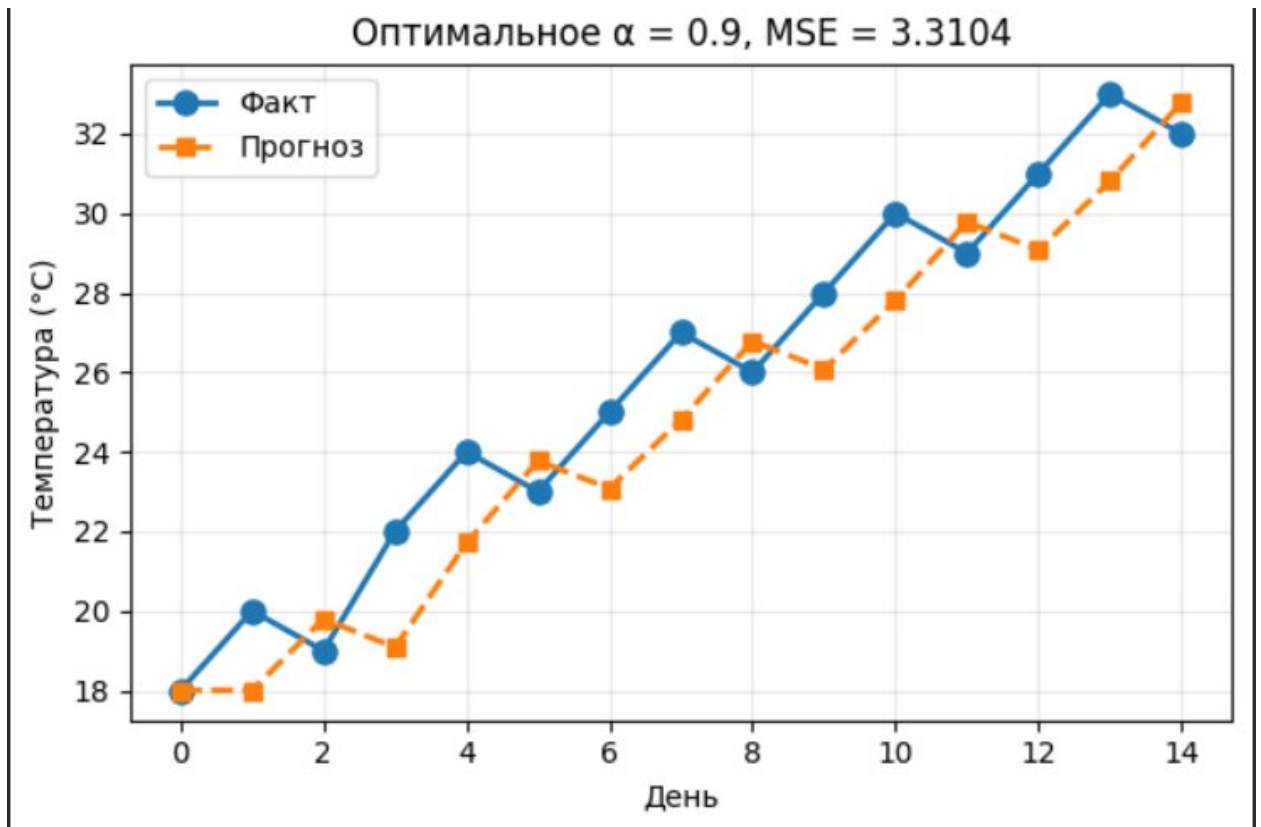
2. Прогноз на 16-й день с $\alpha = 0.9$: 32.08°C

3. Сравнение $\alpha = 0.2$ и оптимального $\alpha = 0.9$:

$\alpha = 0.2$: MSE = 17.5339, Прогноз на 16-й день = 29.03°C

$\alpha = 0.9$: MSE = 3.3104, Прогноз на 16-й день = 32.08°C





Задание 3. Двойное экспоненциальное сглаживание (метод Хольта)

Условие:

Временной ряд имеет линейный тренд. Дан ряд продаж за 12 месяцев:
[100, 110, 115, 125, 135, 140, 150, 160, 165, 175, 185, 190]

Требуется:

1. Реализовать метод двойного экспоненциального сглаживания (метод Хольта):

$$S(t) = \alpha \cdot s(t) + (1-\alpha) \cdot [S(t-1) + T(t-1)]$$

$$T(t) = \beta \cdot [S(t) - S(t-1)] + (1-\beta) \cdot T(t-1)$$

$$\hat{s}(t+k) = S(t) + k \cdot T(t)$$

где $T(t)$ — оценка тренда, β — параметр сглаживания тренда

2. Использовать $\alpha = 0.3$, $\beta = 0.1$

3. Построить прогноз на 3 месяца вперед

4. Вычислить MSE и сравнить с простым экспоненциальным сглаживанием

Решение:

```
sales = [100, 110, 115, 125, 135, 140, 150, 160, 165, 175, 185, 190]
print(f"\nВременной ряд продаж: {sales}")

def holt_method(data, alpha=0.3, beta=0.1):
    n = len(data)
    level = [0] * n
```

```

trend = [0] * n
forecasts = [0] * n
errors = [0] * n

# Инициализация
level[0] = data[0]
trend[0] = data[1] - data[0]
forecasts[0] = data[0]

for t in range(1, n):
    forecasts[t] = level[t-1] + trend[t-1]
    errors[t] = data[t] - forecasts[t]
    level[t] = alpha * data[t] + (1 - alpha) * (level[t-1]
+ trend[t-1])
    trend[t] = beta * (level[t] - level[t-1]) + (1 - beta)
* trend[t-1]

return level, trend, forecasts, errors

# 1. Применение метода Хольта
alpha, beta = 0.3, 0.1
level, trend, forecasts_holt, errors_holt = holt_method(sales,
alpha, beta)
mse_holt = calculate_mse(errors_holt)

print(f"\n1. Метод Хольта с  $\alpha$  = {alpha},  $\beta$  = {beta}")
print(f"MSE = {mse_holt:.4f}")

# 2. Прогноз на 3 месяца вперед
forecast_13 = level[-1] + 1 * trend[-1]
forecast_14 = level[-1] + 2 * trend[-1]
forecast_15 = level[-1] + 3 * trend[-1]

print(f"\n2. Прогноз на 3 месяца вперед:")
print(f"Месяц 13: {forecast_13:.2f}")
print(f"Месяц 14: {forecast_14:.2f}")
print(f"Месяц 15: {forecast_15:.2f}")

# 3. Сравнение с простым экспоненциальным сглаживанием
_, forecasts_simple, errors_simple =
exponential_smoothing(sales, alpha)
mse_simple = calculate_mse(errors_simple)

print(f"\n3. Сравнение с простым экспоненциальным
сглаживанием:")
print(f"Метод Хольта: MSE = {mse_holt:.4f}")
print(f"Простое ЭС: MSE = {mse_simple:.4f}")
print(f"Метод Хольта лучше на {(mse_simple - mse_holt) /
mse_simple * 100}:%")

# График
plt.figure(figsize=(6, 4))
plt.plot(sales, 'o-', label='Факт', linewidth=2, markersize=8)

```

```

plt.plot(forecasts_holt, 's--', label='Метод Хольта',
linewidth=2, markersize=6)
plt.plot(forecasts_simple, '^:', label='Простое ЭС',
linewidth=2, markersize=6)

# Добавляем прогноз на 3 месяца
future_months = [len(sales), len(sales)+1, len(sales)+2]
future_forecasts = [forecast_13, forecast_14, forecast_15]
plt.plot(future_months, future_forecasts, 'ro--',
label='Прогноз', linewidth=2, markersize=8)

plt.xlabel('Месяц')
plt.ylabel('Продажи')
plt.title(f'Метод Хольта vs Простое ЭС ( $\alpha=\{\alpha\}$ ,  $\beta=\{\beta\}$ )')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

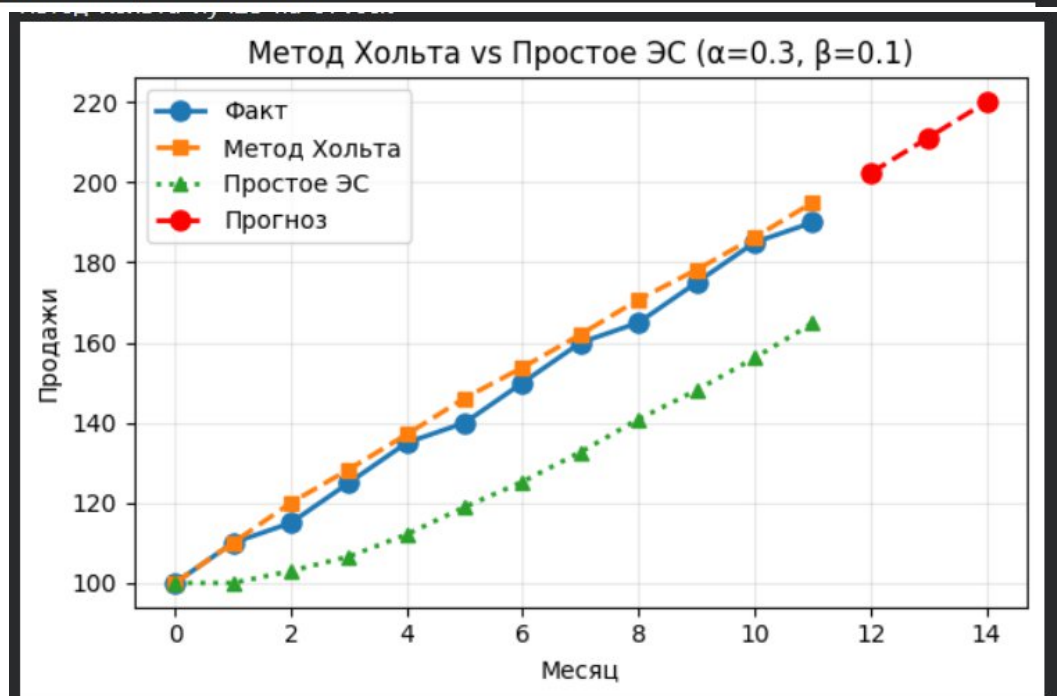
Вывод:

Временной ряд продаж: [100, 110, 115, 125, 135, 140, 150, 160, 165, 175, 185, 190]

1. Метод Хольта с $\alpha = 0.3$, $\beta = 0.1$
MSE = 15.1333

2. Прогноз на 3 месяца вперед:
Месяц 13: 202.32
Месяц 14: 211.19
Месяц 15: 220.06

3. Сравнение с простым экспоненциальным сглаживанием:
Метод Хольта: MSE = 15.1333
Простое ЭС: MSE = 515.5241
Метод Хольта лучше на 97.06%



Задание 4. Анализ сезонности

Условие:

Дан временной ряд продаж мороженого за 24 месяца с явной сезонностью:

[50, 55, 65, 80, 95, 110, 120, 115, 100, 85, 70, 60, 55, 60, 70, 85, 100, 115, 125, 120, 105, 90, 75, 65]

Требуется:

1. Вычислить сезонные индексы (период сезонности = 12 месяцев)
2. Провести декомпозицию ряда на тренд, сезонность и случайную компоненту
3. Построить прогноз на следующие 6 месяцев с учетом сезонности
4. Визуализировать исходный ряд, тренд и сезонную компоненту

Решение:

```
ice_cream = [50, 55, 65, 80, 95, 110, 120, 115, 100, 85, 70, 60, 55, 60, 70, 85, 100, 115, 125, 120, 105, 90, 75, 65]
print(f"\nВременной ряд продаж мороженого: {ice_cream}")

def seasonal_decomposition(data, period=12):
    n = len(data)

    # 1. Вычисление тренда скользящим средним
    trend = [None] * n
    half_period = period // 2

    for t in range(half_period, n - half_period):
        trend[t] = np.mean(data[t-half_period:t+half_period+1])

    # 2. Вычисление сезонной компоненты
    detrended = [data[t] - trend[t] if trend[t] is not None else None for t in range(n)]

    # Вычисляем средние для каждого месяца
    seasonal_indices = []
    for month in range(period):
        month_values = [detrended[i] for i in range(month, n, period) if detrended[i] is not None]
        seasonal_indices.append(np.mean(month_values))

    # Нормализация
    seasonal_indices = np.array(seasonal_indices)
    seasonal_indices -= np.mean(seasonal_indices)

    # 3. Построение полной сезонной компоненты
    seasonal = [seasonal_indices[t % period] for t in range(n)]

    # 4. Случайная компонента
    residual = [data[t] - (trend[t] if trend[t] is not None else np.mean(data)) - seasonal[t]]
```



```

        for t in range(n)]

    return trend, seasonal, residual, seasonal_indices

# 1. Декомпозиция
trend, seasonal, residual, seasonal_indices =
seasonal_decomposition(ice_cream, period=12)

print("\n1. Сезонные индексы (по месяцам):")
for i, idx in enumerate(seasonal_indices, 1):
    print(f"Месяц {i:2d}: {idx:+7.2f}")

# 2. Прогноз на 6 месяцев вперед
# Экстраполяция тренда
valid_trend = [t for t in trend if t is not None]
trend_slope = (valid_trend[-1] - valid_trend[0]) /
len(valid_trend)
last_trend = valid_trend[-1]

print("\n2. Прогноз на следующие 6 месяцев:")
forecasts_seasonal = []
for k in range(1, 7):
    future_trend = last_trend + k * trend_slope
    future_seasonal = seasonal_indices[(len(ice_cream) + k -
1) % 12]
    forecast = future_trend + future_seasonal
    forecasts_seasonal.append(forecast)
    print(f"Месяц {len(ice_cream)+k}: {forecast:.2f}")

# 3. Визуализация
fig, axes = plt.subplots(4, 1, figsize=(6, 12))

# Исходный ряд
axes[0].plot(ice_cream, 'o-', linewidth=2)
axes[0].set_title('Исходный ряд')
axes[0].set_ylabel('Продажи')
axes[0].grid(True, alpha=0.3)

# Тренд
axes[1].plot(trend, 'g-', linewidth=2)
axes[1].set_title('Тренд')
axes[1].set_ylabel('Тренд')
axes[1].grid(True, alpha=0.3)

# Сезонная компонента
axes[2].plot(seasonal, 'r-', linewidth=2)
axes[2].set_title('Сезонная компонента')
axes[2].set_ylabel('Сезонность')
axes[2].grid(True, alpha=0.3)

# Случайная компонента
axes[3].plot(residual, 'k-', linewidth=1)
axes[3].set_title('Случайная компонента')

```

```

axes[3].set_ylabel('Остатки')
axes[3].set_xlabel('Месяц')
axes[3].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

```

Вывод:

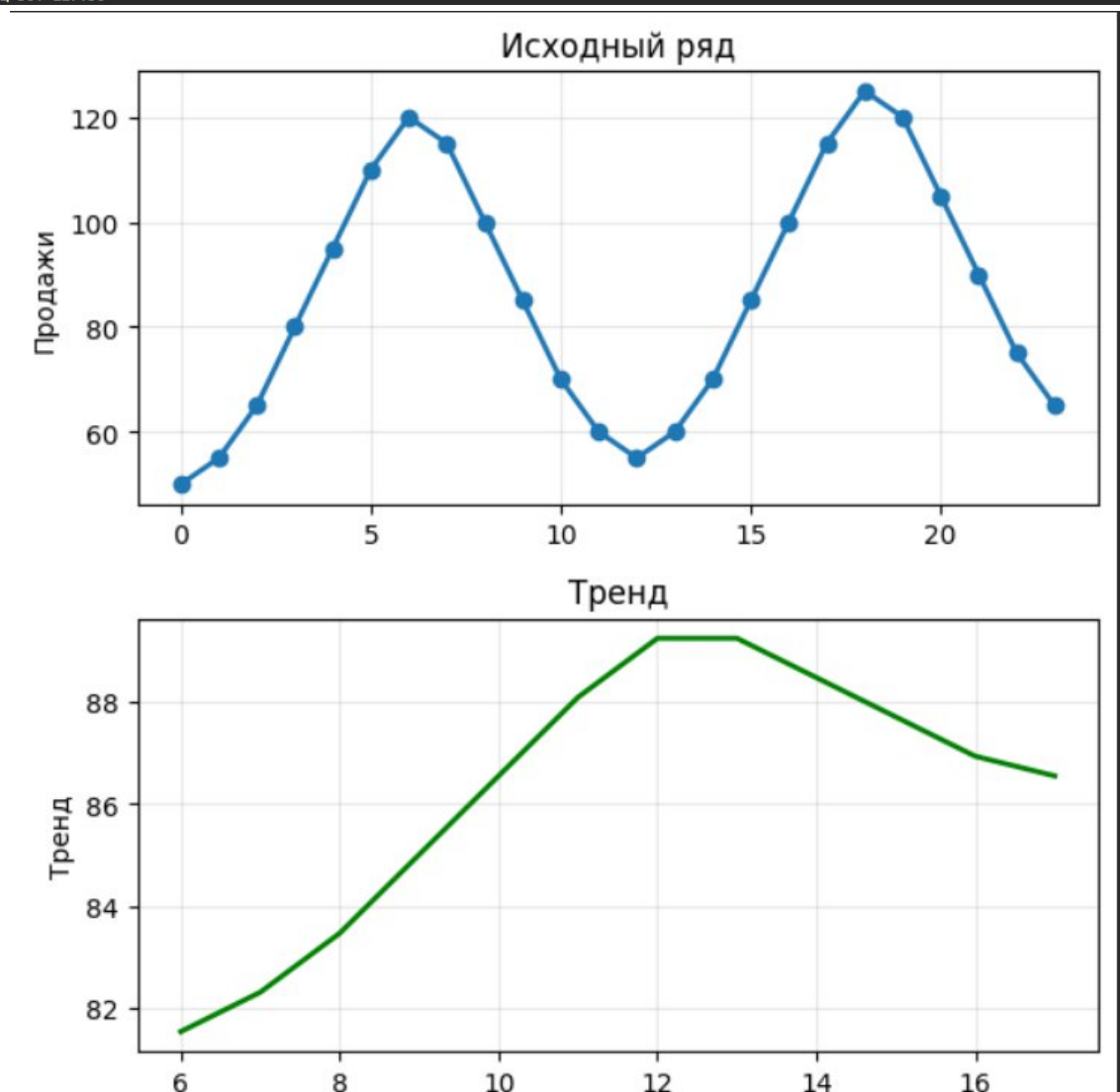
Временной ряд продаж мороженого: [50, 55, 65, 80, 95, 110, 120, 115, 100, 85, 70, 60, 55, 60, 70, 85, 100, 115, 125, 120, 105, 90, 75, 65]

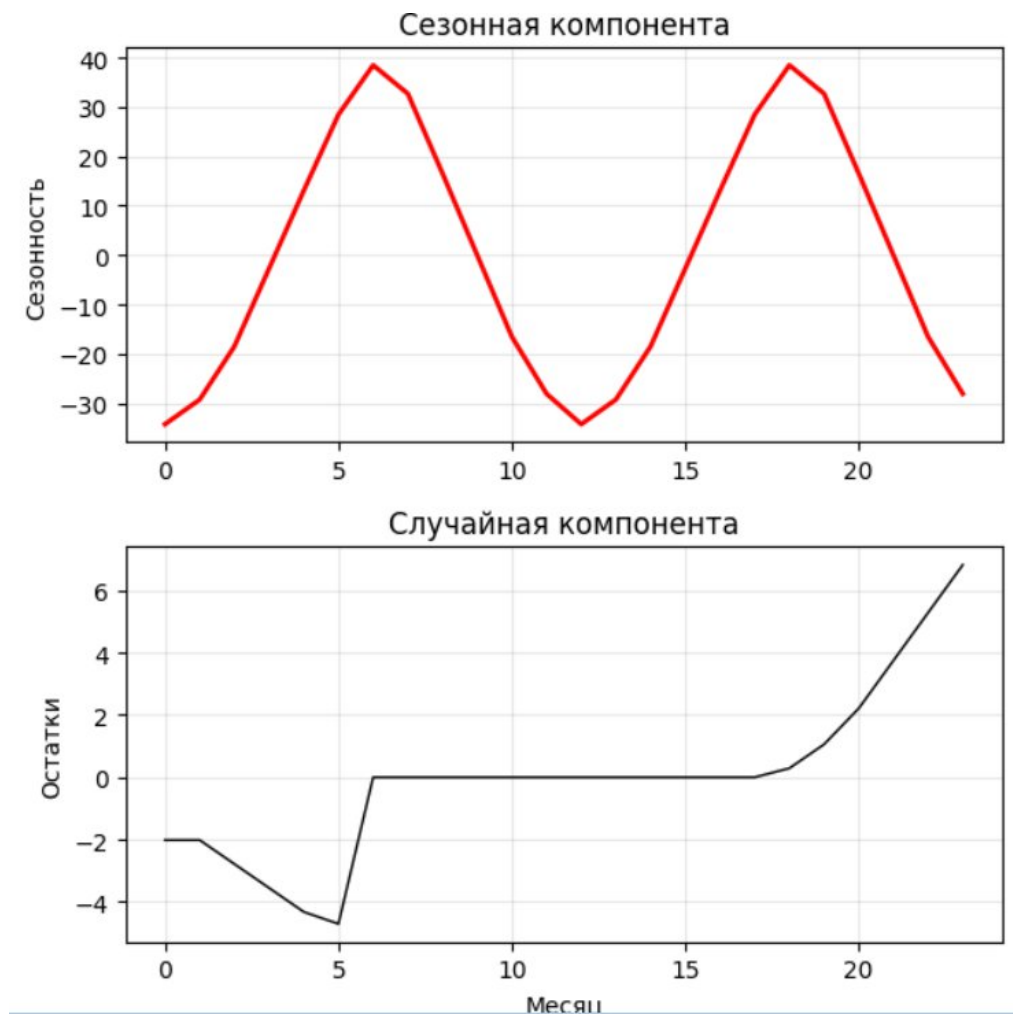
1. Сезонные индексы (по месяцам):

Месяц 1:	-34.23
Месяц 2:	-29.23
Месяц 3:	-18.46
Месяц 4:	-2.69
Месяц 5:	+13.08
Месяц 6:	+28.46
Месяц 7:	+38.46
Месяц 8:	+32.69
Месяц 9:	+16.54
Месяц 10:	-0.00
Месяц 11:	-16.54
Месяц 12:	-28.08

2. Прогноз на следующие 6 месяцев:

Месяц 25:	52.72
Месяц 26:	58.14
Месяц 27:	69.33
Месяц 28:	85.51
Месяц 29:	101.70
Месяц 30:	117.50





Задание 5. Модель авторегрессии AR(1)

Условие:

Дан временной ряд:

[10, 12, 15, 14, 18, 20, 19, 23, 25, 24, 28, 30]

Требуется:

1. Построить модель авторегрессии первого порядка AR(1):

$$s(t) = \varphi_0 + \varphi_1 \cdot s(t-1) + \varepsilon(t)$$

2. Оценить параметры φ_0 и φ_1 методом наименьших квадратов

3. Проверить адекватность модели (проверка остатков на автокорреляцию)

4. Построить прогноз на 3 шага вперед

5. Сравнить с экспоненциальным сглаживанием

Решение:

```
ar_data = [10, 12, 15, 14, 18, 20, 19, 23, 25, 24, 28, 30]
print(f"\nВременной ряд: {ar_data}")

def ar1_model(data):
    # Подготовка данных
    X = np.array(data[:-1]) # s(t-1)
```

```

y = np.array(data[1:])    # s(t)

# МНК: добавляем столбец единиц для  $\phi_0$ 
X_with_const = np.column_stack([np.ones(len(X)), X])

# Решение:  $(X'X)^{-1} X'y$ 
coeffs = np.linalg.lstsq(X_with_const, y, rcond=None)[0]
phi0, phi1 = coeffs

# Вычисление прогнозов и ошибок
forecasts = [data[0]]    # Первое значение без прогноза
errors = [0]

for t in range(1, len(data)):
    forecast = phi0 + phi1 * data[t-1]
    forecasts.append(forecast)
    errors.append(data[t] - forecast)

return phi0, phi1, forecasts, errors

# 1. Оценка параметров
phi0, phi1, forecasts_ar, errors_ar = ar1_model(ar_data)
mse_ar = calculate_mse(errors_ar)

print(f"\n1. Параметры модели AR(1):")
print(f" $\phi_0 = \{phi0:.4f\}$ ")
print(f" $\phi_1 = \{phi1:.4f\}$ ")
print(f"Модель:  $s(t) = \{phi0:.4f\} + \{phi1:.4f\} \cdot s(t-1)$ ")
print(f"MSE =  $\{mse\_ar:.4f\}$ ")

# 2. Прогноз на 3 шага вперед
forecast_13 = phi0 + phi1 * ar_data[-1]
forecast_14 = phi0 + phi1 * forecast_13
forecast_15 = phi0 + phi1 * forecast_14

print(f"\n2. Прогноз на 3 шага вперед:")
print(f" $s(13) = \{forecast\_13:.2f\}$ ")
print(f" $s(14) = \{forecast\_14:.2f\}$ ")
print(f" $s(15) = \{forecast\_15:.2f\}$ ")

# 3. Сравнение с экспоненциальным сглаживанием
_, forecasts_es, errors_es = exponential_smoothing(ar_data, 0.3)
mse_es = calculate_mse(errors_es)

print(f"\n3. Сравнение с экспоненциальным сглаживанием:")
print(f"AR(1): MSE =  $\{mse\_ar:.4f\}$ ")
print(f"ЭС ( $\alpha=0.3$ ): MSE =  $\{mse\_es:.4f\}$ ")

# График
plt.figure(figsize=(6, 6))
plt.plot(ar_data, 'o-', label='Факт', linewidth=2, markersize=8)
plt.plot(forecasts_ar, 's--', label='AR(1)', linewidth=2, markersize=6)

```

```
plt.plot(forecasts_es, '^:', label='ЭС', linewidth=2,
markersize=6)
plt.xlabel('Время')
plt.ylabel('Значение')
plt.title('Модель AR(1) vs Экспоненциальное сглаживание')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

Вывод:

Временной ряд: [10, 12, 15, 14, 18, 20, 19, 23, 25, 24, 28, 30]

1. Параметры модели AR(1):

$\phi_0 = 2.7429$

$\phi_1 = 0.9511$

Модель: $s(t) = 2.7429 + 0.9511 \cdot s(t-1)$

MSE = 3.5314

2. Прогноз на 3 шага вперед:

$s(13) = 31.28$

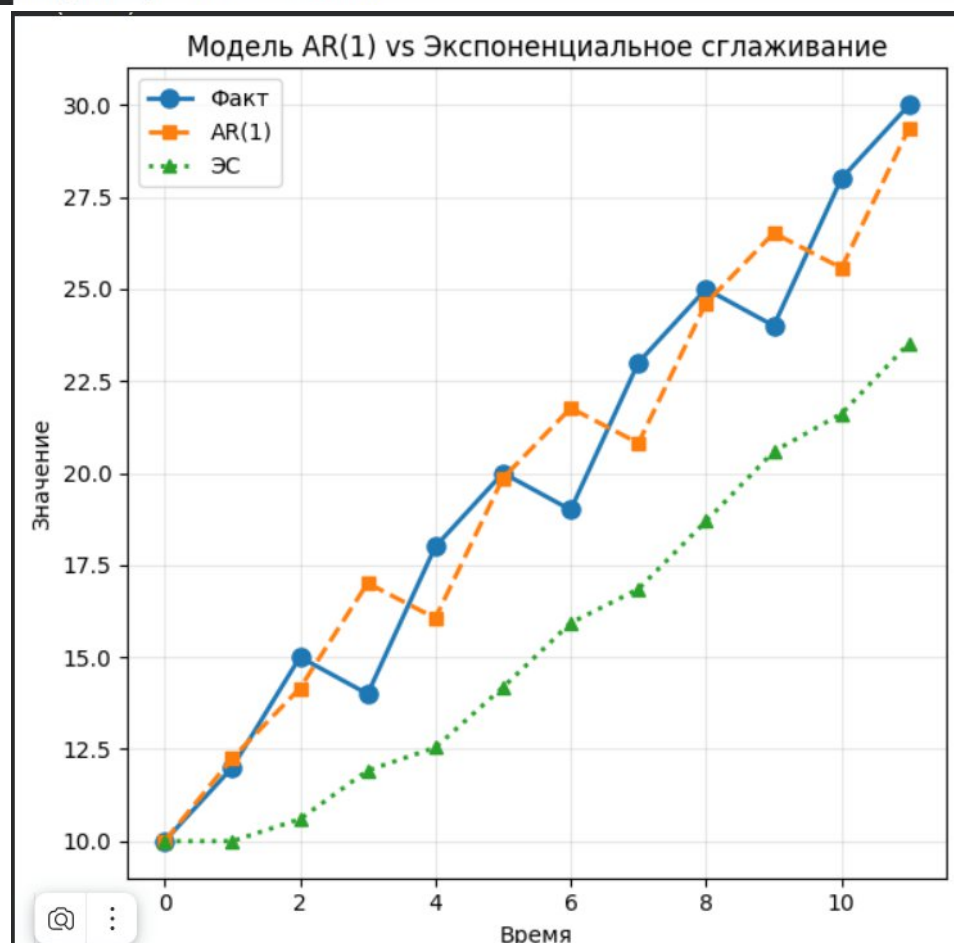
$s(14) = 32.49$

$s(15) = 33.64$

3. Сравнение с экспоненциальным сглаживанием:

AR(1): MSE = 3.5314

ЭС ($\alpha=0.3$): MSE = 24.7960



Задание 6. Комбинированное прогнозирование

Условие:

Дан временной ряд цен акций за 20 дней:

[100, 102, 105, 103, 107, 110, 108, 112, 115, 113, 117, 120, 118, 122, 125, 123, 127, 130, 128, 132]

Требуется:

1. Построить три модели прогнозирования:
 - Экспоненциальное сглаживание ($\alpha = 0.3$)
 - Скользящее среднее (окно = 3)
 - Линейная регрессия
2. Для каждой модели вычислить MSE, MAE, MAPE
3. Построить комбинированный прогноз как взвешенное среднее трех моделей (веса пропорциональны обратным MSE)
4. Сравнить качество комбинированного прогноза с отдельными моделями
5. Построить прогноз на 5 дней вперед

Решение:

```
stocks = [100, 102, 105, 103, 107, 110, 108, 112, 115, 113,
          117, 120, 118, 122, 125, 123, 127, 130, 128, 132]
print(f"\nВременной ряд цен акций: {stocks}")

def moving_average(data, window=3):
    n = len(data)
    forecasts = [data[0]] * window # Первые значения без прогноза
    errors = [0] * window

    for t in range(window, n):
        forecast = np.mean(data[t-window:t])
        forecasts.append(forecast)
        errors.append(data[t] - forecast)

    return forecasts, errors

def linear_regression_forecast(data):
    n = len(data)
    X = np.arange(n)
    y = np.array(data)

    # МНК для линейной регрессии
    X_with_const = np.column_stack([np.ones(n), X])
    coeffs = np.linalg.lstsq(X_with_const, y, rcond=None)[0]
    a, b = coeffs

    forecasts = [a + b * i for i in range(n)]
    errors = [data[i] - forecasts[i] for i in range(n)]

    return forecasts, errors, a, b
```

```

# 1. Три модели прогнозирования
print("\n1. Три модели прогнозирования:")

# Модель 1: Экспоненциальное сглаживание
_, forecasts_es1, errors_es1 = exponential_smoothing(stocks,
0.3)
mse_es1 = calculate_mse(errors_es1)
mae_es1 = np.mean(np.abs(errors_es1[1:]))
mape_es1 = np.mean(np.abs(np.array(errors_es1[1:]))) /
np.array(stocks[1:])) * 100

print(f"Экспоненциальное сглаживание ( $\alpha=0.3$ ):")
print(f"    MSE = {mse_es1:.4f}, MAE = {mae_es1:.4f}, MAPE =
{mape_es1:.4f}%")

# Модель 2: Скользящее среднее
forecasts_ma, errors_ma = moving_average(stocks, window=3)
mse_ma = calculate_mse(errors_ma)
mae_ma = np.mean(np.abs(errors_ma[3:]))
mape_ma = np.mean(np.abs(np.array(errors_ma[3:]))) /
np.array(stocks[3:])) * 100

print(f"Скользящее среднее (окно=3):")
print(f"    MSE = {mse_ma:.4f}, MAE = {mae_ma:.4f}, MAPE =
{mape_ma:.4f}%")

# Модель 3: Линейная регрессия
forecasts_lr, errors_lr, a, b =
linear_regression_forecast(stocks)
mse_lr = calculate_mse(errors_lr)
mae_lr = np.mean(np.abs(errors_lr[1:]))
mape_lr = np.mean(np.abs(np.array(errors_lr[1:]))) /
np.array(stocks[1:])) * 100

print(f"Линейная регрессия:")
print(f"    MSE = {mse_lr:.4f}, MAE = {mae_lr:.4f}, MAPE =
{mape_lr:.4f}%")

# 2. Комбинированный прогноз
print("\n2. Комбинированный прогноз:")

# Веса пропорциональны обратным MSE
w_es = (1 / mse_es1)
w_ma = (1 / mse_ma)
w_lr = (1 / mse_lr)
total = w_es + w_ma + w_lr

w_es /= total
w_ma /= total
w_lr /= total

print(f"Веса моделей:")
print(f"    ЭС: {w_es:.4f}")

```

```

print(f"    CC: {w_ma:.4f}")
print(f"    ЛР: {w_lr:.4f}")

# Комбинированные прогнозы
forecasts_combined = []
for i in range(len(stocks)):
    combined = w_es * forecasts_es1[i] + w_ma * forecasts_ma[i]
    + w_lr * forecasts_lr[i]
    forecasts_combined.append(combined)

errors_combined = [stocks[i] - forecasts_combined[i] for i in
range(len(stocks))]
mse_combined = calculate_mse(errors_combined)
mae_combined = np.mean(np.abs(errors_combined[1:]))
mape_combined = np.mean(np.abs(np.array(errors_combined[1:]) /
np.array(stocks[1:]))) * 100

print(f"\nКомбинированная модель:")
print(f"    MSE = {mse_combined:.4f}, MAE = {mae_combined:.4f},
MAPE = {mape_combined:.4f}%")

# 3. Прогноз на 5 дней вперед
print("\n3. Прогноз на 5 дней вперед:")

# Для ЭС
last_smoothed_es = forecasts_es1[-1]
future_es = [last_smoothed_es] * 5

# Для CC
future_ma = [np.mean(stocks[-3:])] * 5

# Для ЛР
n = len(stocks)
future_lr = [a + b * (n + k) for k in range(1, 6)]

# Комбинированный
future_combined = []
for k in range(5):
    combined = w_es * future_es[k] + w_ma * future_ma[k] + w_lr
    * future_lr[k]
    future_combined.append(combined)
    print(f"День {len(stocks)+k+1}: {combined:.2f}")

# График
plt.figure(figsize=(7, 6))
plt.plot(stocks, 'o-', label='Факт', linewidth=2, markersize=8)
plt.plot(forecasts_es1, 's--', label='ЭС', linewidth=1,
markersize=4, alpha=0.7)
plt.plot(forecasts_ma, '^--', label='CC', linewidth=1,
markersize=4, alpha=0.7)
plt.plot(forecasts_lr, 'v--', label='ЛР', linewidth=1,
markersize=4, alpha=0.7)
plt.plot(forecasts_combined, 'd-', label='Комбинированная',

```



```

linewidth=2, markersize=6)

# Прогноз на 5 дней
future_days = list(range(len(stocks), len(stocks) + 5))
plt.plot(future_days, future_combined, 'ro--', label='Прогноз',
linewidth=2, markersize=8)

plt.xlabel('День')
plt.ylabel('Цена акции')
plt.title('Комбинированное прогнозирование')
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

```

Вывод:

Временной ряд цен акций: [100, 102, 105, 103, 107, 110, 108, 112, 115, 113, 117, 120, 118, 122, 125, 123, 127, 130, 128, 132]

1. Три модели прогнозирования:

Экспоненциальное сглаживание ($\alpha=0.3$):

MSE = 27.4177, MAE = 4.8541, MAPE = 4.1131%

Скользящее среднее (окно=3):

MSE = 11.3158, MAE = 3.1961, MAPE = 2.6899%

Линейная регрессия:

MSE = 2.1702, MAE = 1.2724, MAPE = 1.0962%

2. Комбинированный прогноз:

Веса моделей:

ЭС: 0.0623

СС: 0.1509

ЛР: 0.7868

Комбинированная модель:

MSE = 2.8701, MAE = 1.5712, MAPE = 1.3469%

3. Прогноз на 5 дней вперед:

День 21: 133.54

День 22: 134.84

День 23: 136.14

День 24: 137.43

День 25: 138.73

